

FRAUD DETECTION PLATFORM · AWS

Fraud Detection Platform

Real-time fraud detection at scale on AWS — extreme class imbalance, agentic investigation, minimal customer friction.

SageMaker

EKS

EMR / Spark

Airflow

Claude (Bedrock)

Databricks · MLflow

Staff ML Engineer — project deep-dive

What this deck covers

1

The journey

Discovery to delivery

2

Architecture

Full AWS stack + cloud services

3

ML training

Airflow, EMR/Spark, SageMaker

4

Agents + RAG

4 agents, OpenSearch k-NN

5

Lifecycle & release

Experiment to prod, safely

6

Reliability

Agent, data/ML & system failures

The problem we set out to solve

Business context

Fraud losses were rising and the existing rules engine was both leaky and noisy — too much fraud got through while too many genuine customers were wrongly blocked. The mandate: score transactions in near-real-time, at the scale of millions, cutting losses without adding customer friction.

The shape of the solution

A hybrid ML scorer (supervised + unsupervised) on a streaming pipeline, with four investigation agents that triage alerts, run AML checks, analyse finances and audit security — surfacing a grounded case summary to fraud analysts, all on AWS.

~0.1%

fraud base rate — extreme imbalance drove every metric choice

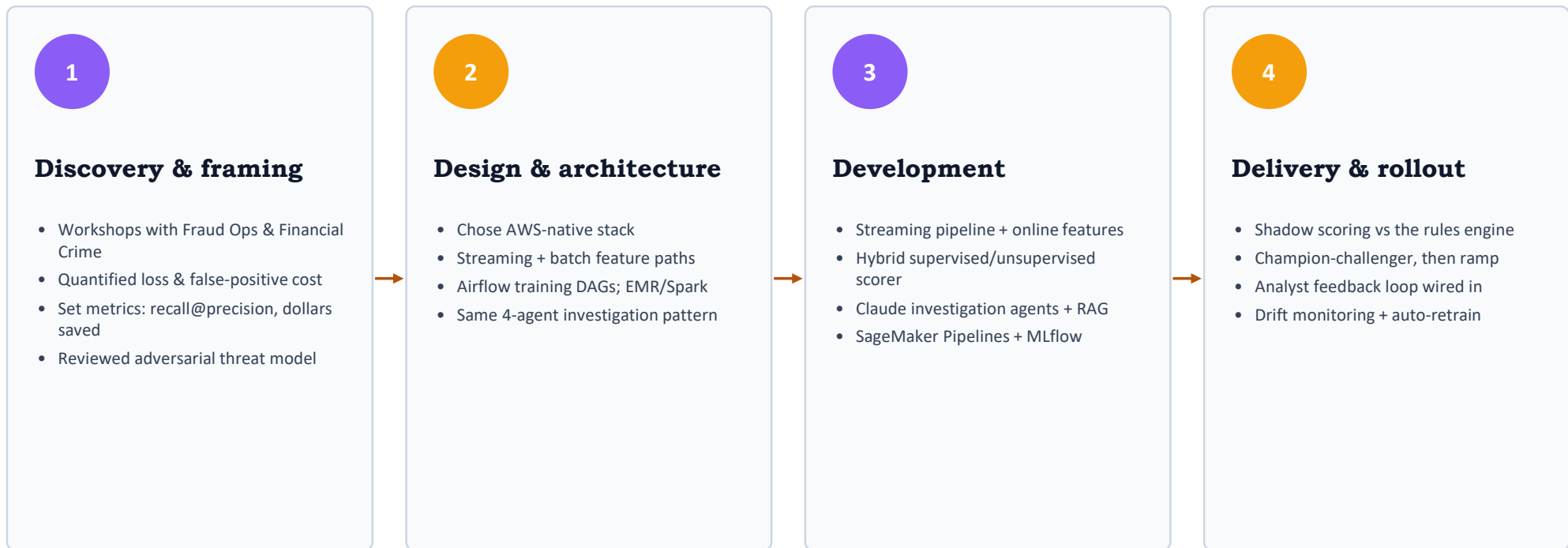
Near-real-time

sub-second scoring on streaming transactions at scale

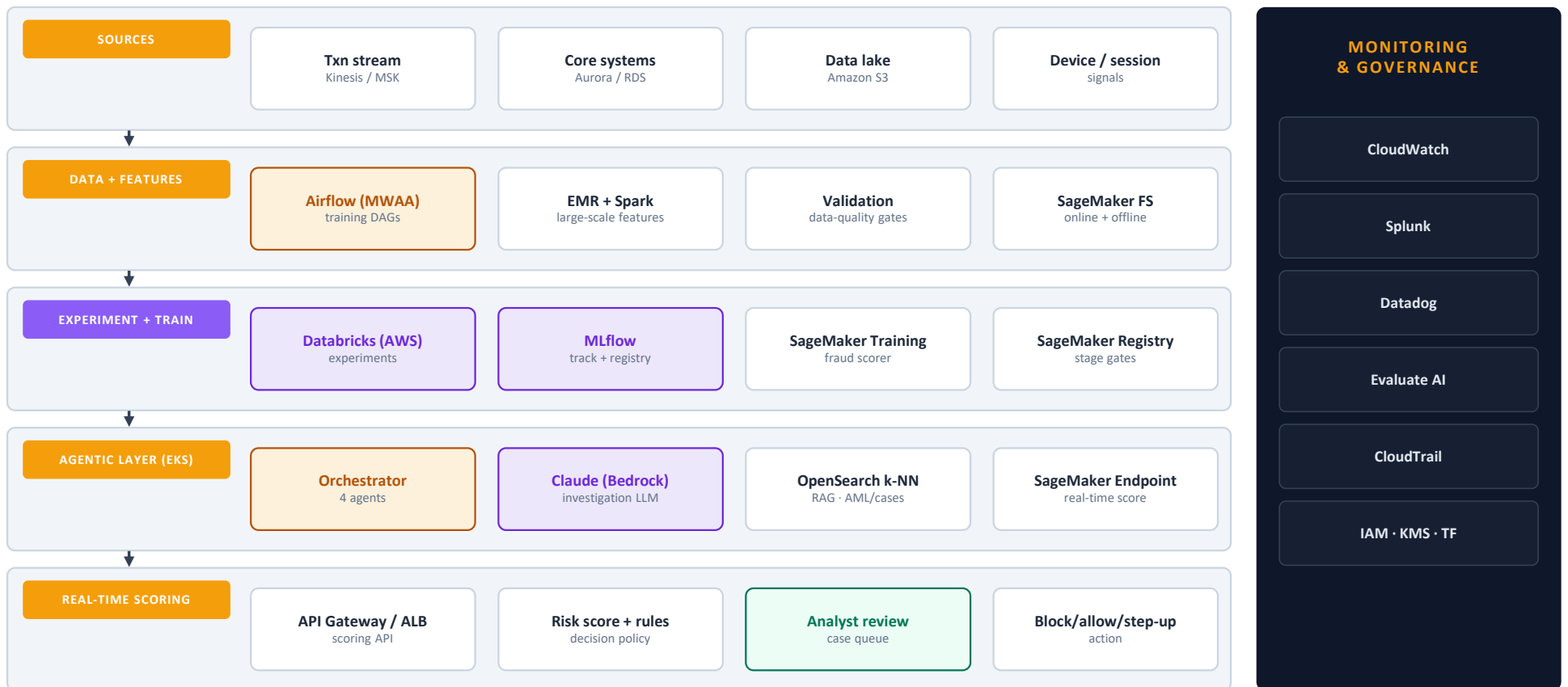
Analyst-in-loop

agents draft the case; humans decide the edge

How it unfolded: discovery to delivery



Solution architecture — end to end with cloud services



AWS services used across the platform

● Compute & Orchestration

Amazon EKS
Airflow (Amazon MWAA)
AWS Lambda

● Data & Analytics

Amazon S3
Kinesis · MSK (Kafka)
EMR (Spark) · Glue
Athena · Redshift
DynamoDB

● ML Platform — SageMaker

SageMaker Training
SageMaker Endpoints
SageMaker Feature Store
SageMaker Model Registry
SageMaker Pipelines

● LLM & RAG

Claude on Amazon Bedrock
Amazon OpenSearch (k-NN)
Bedrock Knowledge Bases

● CI/CD & IaC

CodePipeline / CodeBuild
Amazon ECR
Terraform

● Security & Governance

IAM · KMS
VPC · Secrets Manager
CloudTrail

● Monitoring & Alerting

Amazon CloudWatch
Splunk · Datadog
Evaluate AI (LLM eval)

How model training works — with cloud services



Orchestrated by Airflow (MWAA) + SageMaker Pipelines — every run idempotent, versioned, reproducible (MLflow)

Retraining is triggered, not scheduled by habit. Drift detectors and the analyst feedback loop trigger retraining; champion-challenger validates a new model in shadow before it ever takes traffic.

Imbalance-aware

PR-AUC & recall-at-fixed-precision, resampling on the train fold only — never raw accuracy.

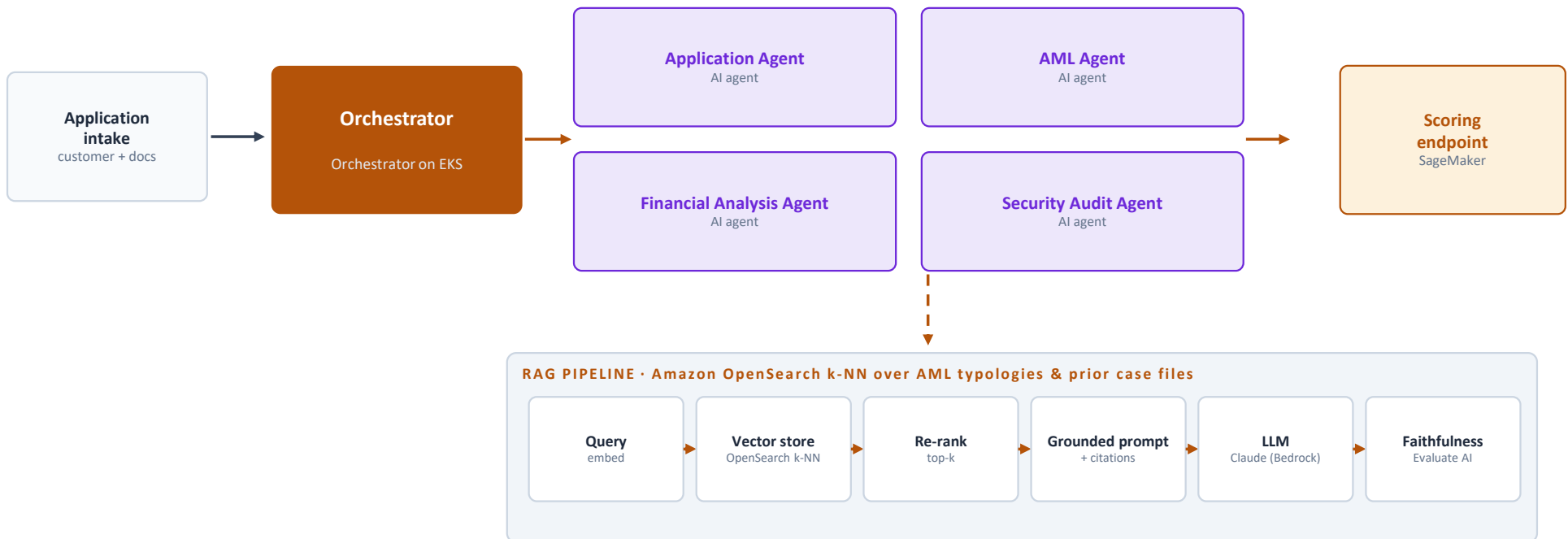
Distributed at scale

EMR/Spark computes billions of behavioural & velocity features; the online store serves them sub-second.

Delayed labels

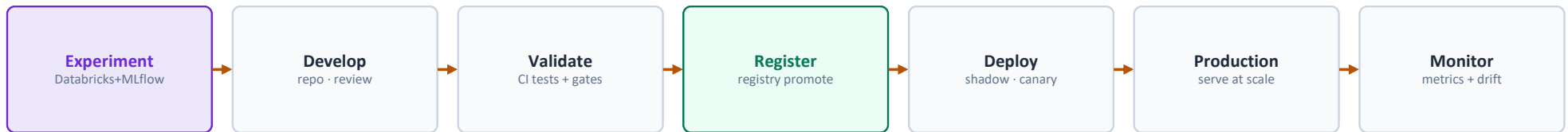
Fraud is confirmed late, so the pipeline handles label lag & weak labels explicitly.

AI agents + RAG pipeline — with cloud services



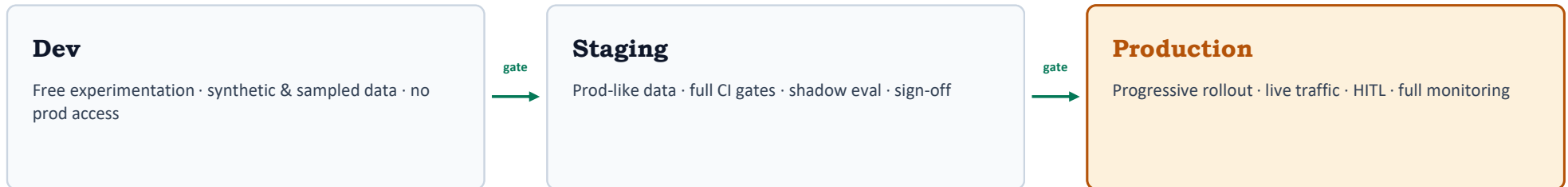
Agents call the ML scoring endpoint + data tools, then return a grounded, cited recommendation -> reason codes -> human-in-the-loop for the edge cases.

Experiments -> development -> deployment -> production



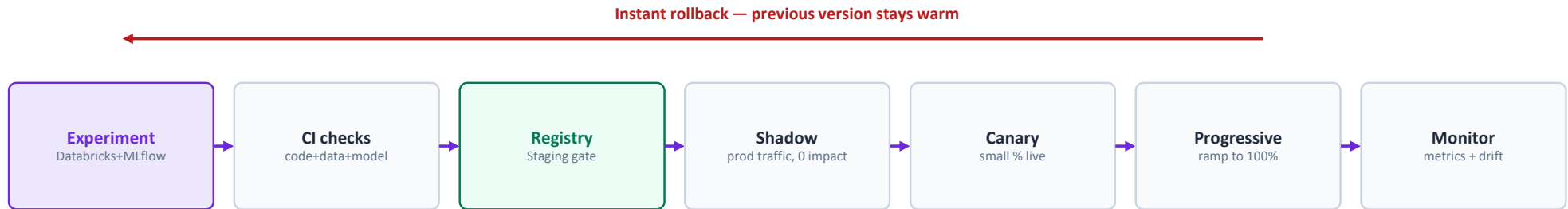
Monitoring closes the loop — drift / decay opens the next experiment

PROMOTION ACROSS ENVIRONMENTS

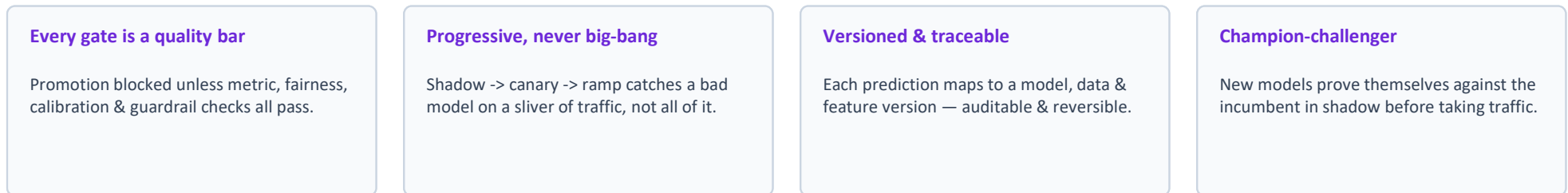


Each promotion is gated; artifacts (code, data snapshot, feature & model versions) are immutable and traceable end to end via MLflow + the model registry.

Shipping fast without compromising production



Drift / decay -> triggers a new experiment (closed loop)



Monitoring & measuring ML metrics and accuracy



Operational health

CloudWatch + Datadog

Latency p50/p99, throughput, error rate, saturation, cost per 1k predictions



Data & feature drift

PSI / KS monitors

Input & prediction-distribution shift, feature freshness & null spikes



Model performance

Sliced metrics

PR-AUC, recall@precision, dollars of fraud caught, false-positive rate — by segment



LLM / agent quality

Evaluate AI

Faithfulness, citation validity, refusal / leak checks on every agent output

Splunk aggregates logs & fires alerts across all four layers. Alert thresholds are tied to business impact, not arbitrary statistics. Because fraud labels confirm late, ground-truth accuracy is tracked with deliberate delayed-label evaluation, while real-time proxies (score distribution, analyst hit-rate) catch problems early.

ML development strategy — MLflow, TensorFlow & PyTorch

MLflow — the connective tissue. One tracking + registry layer across frameworks and clouds: every run's params, data & feature versions, metrics and artifacts are logged; the registry governs Staging-to-Production promotion with the release gates. Runs on Databricks; models serve on SageMaker.

PyTorch · research & iteration

- Fast, flexible experimentation — default for research & rapid iteration
- Where we explore sequence / graph models for fraud patterns
- Dynamic graphs make debugging & custom logic easy
- Served via TorchServe or exported to ONNX / SageMaker

TensorFlow · production serving

- Mature serving & lifecycle ecosystem (TF Serving, TFX)
- Preferred for a hardened, high-throughput serving path
- Strong fit for managed SageMaker endpoints
- Graph optimisation & quantisation for sub-second inference

The strategy in one line: *prototype freely in PyTorch, standardise artifacts & metrics through MLflow, then serve on SageMaker — and a gradient-boosted scorer plus unsupervised anomaly detection still beats a deep net on this tabular, imbalanced problem.*

What I owned vs. what the team delivered

What I personally owned · Staff scope

- Owned the end-to-end architecture & system-design-at-scale decisions
- ML development & deployment: scorer, thresholding, evaluation, SageMaker serving
- AI/agent development & deployment: orchestration, tool contracts, guardrails
- Designed the EMR/Spark feature pipeline + SageMaker online feature store
- Set imbalance-aware evaluation & threshold-tuning methodology
- Designed champion-challenger release & the analyst feedback loop
- Reused & hardened the agent + RAG framework; mentored engineers across both clouds

What the team delivered

- Built streaming ingestion & Spark feature jobs to spec
- Implemented model training & the agent tools
- Ran EKS platform, autoscaling & on-call
- Fraud analysts labelled cases & tuned review thresholds with us
- SecOps integrated detection with incident response
- QA executed load & latency testing at scale

Maps to the role:

Architecture

ML dev & deploy

AI dev & deploy

System design

Standards

Mentoring

Client & stakeholder intersection



Fraud Operations

Co-designed the alert queue & case summaries with the analysts who work them daily.



Financial Crime / AML

Aligned detection & agent reasoning with AML typologies and reporting obligations.



Customer Experience

Tuned thresholds against false-positive friction — protecting genuine customers from blocks.



Risk & Compliance

Presented model validation & monitoring evidence for governance sign-off.



Data Platform / SRE

Agreed streaming SLAs, autoscaling & cost guardrails for real-time scoring.



SecOps

Wired detections into incident response & the security audit agent's signals.

Technology decisions & trade-offs

Decision	What I chose	Trade-off I accepted
Model approach	Hybrid supervised + unsupervised	More complexity, but catches both known fraud and novel / zero-day patterns
Feature compute	EMR/Spark over AWS Glue	More cluster ops, bought heavy distributed throughput & spot-instance savings
Training & serving	SageMaker managed	Some lock-in, in exchange for managed pipelines, registry & autoscaled endpoints
Orchestration	Airflow (MWAA) over Step Functions	Self-managed feel, but richer DAGs & portability across our stack
Decision action	Step-up auth, not just block	More flows to build, but far less customer friction than hard blocks

Challenges & how I worked through them

Extreme class imbalance (~0.1%)

Anchored on PR-AUC & recall-at-fixed-precision tuned to the cost matrix; resampled only inside the training fold; calibrated scores.

Sub-second latency at scale

Precomputed heavy features in the online store, computed only light features live, cached aggressively, autoscaled on EKS with graceful fallback.

Adversaries & concept drift

Drift monitors + champion-challenger + frequent triggered retraining so the model keeps pace with evolving fraud.

False positives hurt real customers

Tuned thresholds to analyst capacity, added step-up auth, fed analyst decisions back as labels to sharpen precision.

Addressing AI & agent failure modes

● Hallucination / unfaithful output

How we contain it: ground every claim via RAG with citations over case files; verify figures against source data; an Evaluate AI faithfulness gate + confidence threshold blocks risky output; low confidence escalates to an analyst.

● Bad arguments passed to tools

How we contain it: strict typed tool schemas with JSON-schema validation; arguments validated & sanitised before any call; auto-repair retry on failure; the agent asks rather than guesses on ambiguity.

● Wrong tool selected

How we contain it: task-scoped tool allow-lists; clear tool specs + few-shot; a planner-critic review step; deterministic routing where rules suffice; tool-choice accuracy tracked in eval.

● Context overload

How we contain it: retrieve, don't stuff — top-k relevant case context only; rolling summarisation of the investigation; structured memory; hard per-agent token budgets with relevance filtering.

Handling data / ML & system / tool failures

DATA / ML

Data-quality drift

Schema + distribution checks (PSI/KS) on the stream & batch paths; anomaly detection; a circuit-breaker halts promotion & serving on bad data.

DATA / ML

Concept drift

Adversaries evolve fast — label-based monitors + drift detectors trigger retraining; champion-challenger before cutover.

DATA / ML

Missing feature values

SageMaker Feature Store with versioned defaults & imputation; null-rate monitoring; safe fallbacks keep scoring online in a flagged degraded mode.

SYSTEM / TOOL

API timeouts

Timeouts + exponential-backoff retries, circuit breakers, bulkheads, caching and async calls with a graceful fallback path.

SYSTEM / TOOL

Schema mismatches

Schema registry + contract tests in CI; versioned, backward-compatible APIs; validation at every boundary.

SYSTEM / TOOL

Orchestration bottlenecks

Parallelise independent agents; concurrency limits + backpressure; rate limiting; idempotent steps; distributed tracing to find hot spots.

WHAT THIS JOURNEY SHOWS

The through-line

● Production ML at scale

Online feature store, sub-second SageMaker serving, drift-aware retraining at millions of transactions.

● Principled trade-offs

Imbalance-aware metrics, step-up over hard blocks, managed vs self-managed on merit.

● Resilient by design

Concrete containment for agent, data/ML and system failures — built for adversaries.

● Standards & mentoring

One agent + RAG pattern reused across two clouds; I grew the engineers who built it.

Real-time ML and agentic AI at scale, protecting customers without getting in their way — the work I want to keep doing.